

1 Closest Pair of Points

In this problem, we will be computing the closest pair of points among one million points on the 2D plane using spatial hashing. Finding the closest pair of points has many applications; in machine learning classification, hierarchical clustering, collision detection in games and more.

The input is given in the contains at most one million points in the two dimensional space, each point described by its x and y coordinates on the unit square (each x and y coordinate lies in $[0, 1]$). Each point is described on a single line using two space separated real values. The Euclidean distance between two points (x_1, y_1) and (x_2, y_2) is given by the following formula:

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The naïve solution to this problem is to compute the distance between every pair of points, and always keeping track of the minimum distance computed. This takes $\Theta(n^2)$ time, and is infeasible in our case, where $n = 1,000,000$. Instead, we divide the unit square into $b \times b$ grids, each of size $\frac{1}{b} \times \frac{1}{b}$. Each point is therefore hashed into its specific grid (How?), and need to be only compared against points in its grid and the neighbouring 8 grids. This significantly reduces the number of pairs of points we look, provided we choose a good grid size. Feel free to experiment with different choices for the value of b .

Each of these grid cells should contain a list to store all the points hashed into it. Your program should roughly follow these steps:

1. Allocate a 2D array of grid cells, for a particular choice of b .
2. Read the input, and hash each point to its corresponding grid cell.
3. For each point, compare against all points in its grid cell and neighbouring grid cells, always maintaining the minimum distance computed.
4. Print the minimum distance.

As output, you only need to print a single real number that is the distance between the closest pair of points in the input. A sample input containing 10^6 points is given in `closest.in`. It's coutput is in `closest.out`.

2 Song Key

You are developing a smartphone app that converts audio recorded from an instrument into sheet music. You already have the code to record the audio and obtain a sequence of frequency values.

What is left is to translate those frequencies into pitch names more easily recognizable to musicians.

2.1 Some Music Theory

Musical pitches are described by frequencies using a logarithmic scale. The frequency 440Hz corresponds to the pitch named *A*. Multiplying a pitch's frequency by $\sqrt[12]{2}$ produces the next higher pitch. The table below lists pitches used in Western music and their frequencies.

Frequency (in Hz)	Pitch name
440	A
$440 \cdot 2^{1/12} \approx 466.16$	$A\sharp, B\flat$
$440 \cdot 2^{2/12} \approx 493.88$	B
$440 \cdot 2^{3/12} \approx 523.25$	C
$440 \cdot 2^{4/12} \approx 554.37$	$C\sharp, D\flat$
$440 \cdot 2^{5/12} \approx 587.37$	D
$440 \cdot 2^{6/12} \approx 622.25$	$D\sharp, E\flat$
$440 \cdot 2^{7/12} \approx 659.26$	E
$440 \cdot 2^{8/12} \approx 698.46$	F
$440 \cdot 2^{9/12} \approx 739.99$	$F\sharp, G\flat$
$440 \cdot 2^{10/12} \approx 783.99$	G
$440 \cdot 2^{11/12} \approx 830.61$	$G\sharp, A\flat$
880	A

Doubling any pitch's frequency yields the same pitch, but one octave higher (note that both 440 Hz and 880 Hz correspond to pitch A). This rule generalizes: multiplying 554.37 Hz by a power of two yields another $C\sharp$. Some pitches have multiple names, as shown in the table.

2.2 Your Task

You have a song, represented as a sequence of frequencies. Songs in a musical key typically contain only a subset of the possible pitches. Assume that the given song is in one of the common keys listed in table below. First, determine what key the song is in. Then, translate the song into a sequence of pitch names.

Key name	Allowed pitches
G major	$G, A, B, C, D, E, F\sharp$
C major	C, D, E, F, G, A, B
$E\flat$ major	$E\flat, F, G, A\flat, B\flat, C, D$
$F\sharp$ minor	$F\sharp, G\sharp, A, B, C\sharp, D, E$
G minor	$G, A, B\flat, C, D, E\flat, F$

2.3 Input and Output

The first line of input contains a single integer $1 \leq n \leq 100,000$, the number of frequencies in the song. The next n lines each contain a real number $20 \leq f \leq 4000$, representing one pitch frequency, with at most 8 digits after the decimal point. You may assume that the song was played using an instrument that is in tune, so that each frequency represents a valid pitch: each frequency approximates $440 \cdot 2^{\alpha/12}$ for some integer α with absolute error at most 10^{-4} .

On the first line of the output, print the name of the key that the song was played in: G major, $F\sharp$ minor, etc. See below for instructions for how to format pitch names. If the song does not fit any of the keys in the table above, or if multiple possible keys could fit the song, print “cannot determine key”.

If the key could not be determined, print no further output. Otherwise, print n lines, each containing one pitch name (in the order of the input sequence). Each pitch name should be a letter A through G followed optionally by a second character, either $\sharp$ or b , indicating a sharp or flat. Note that although pitches can have multiple names, only one of these names is listed in the allowed-pitches table for any given key, and this is the name you should use. For example, print $C\sharp$, and not $D\flat$, when writing pitches for the $F\sharp$ key minor. Sample input and outputs are given below.

Sample Input:

```
7
783.9908720
783.9908720
587.3295358
587.3295358
659.2551138
659.2551138
587.3295358
```

Sample Output:

```
cannot determine key
```

Sample Input:

```
5
523.2511306
391.9954360
440.0
329.6275569
369.9944227
```

Sample Output:

```
G major
C
G
A
E
F#
```

3 Postamble

3.1 Input and Output

The input for all programs will always be accepted from standard input (keyboard or `system.in`) and output will always be sent to standard output (monitor or `system.out`). If the input contains many lines, then you can put your input in a separate file, say `progrname.in` and re-direct this input into your program by running

```
python3 progrname.py < progrname.in
```

You could also redirect the output of your program to another file, say `myoutput.txt`.

```
python3 progrname.py < progrname.in > myoutput.txt
```

If the intended output is in file `progrname.out`, then you can use the `diff` utility to check your output by taking the difference of two files.

```
python3 progrname.py < progrname.in > myoutput.txt
diff myoutput.txt progrname.out
```

If the output of the `diff` command returns nothing, this means that your output is the same as the intended output, and is judged correct for a test case. Note that your program is only correct if it is judged correct for all test cases. You are encouraged to design different test cases and test the correctness of your programs thoroughly.

Please do not include extraneous input or output as these would throw off our automated grading scripts. The input and the output should always match the format given in the description of each problem. Sample test cases are provided for you to test the formatting of the input and output, as well as to check the correctness of your program for simple test cases. Note that we will be testing with a large number of test cases.

3.2 Submission

Please submit all the individual files to **Canvas**. Please name your programs as given in the problem description, or refer to the names in the following table.

Problem Name	File Name
Closest Pair of Points	closest.py
Song key	song-key.py